

boaz_학기 세션 과제(3)

GAN

1. Abstract
2. Introduction
 - 2.1. 기존 딥러닝 모델
 - 2.2. Adversarial Nets 제안
 - 2.3. Advantages of GAN
3. Methods
 - 3.1. Adversarial nets
 - 3.2. Theoretical Results
4. Experiments
 - 4.1. Dataset
 - 4.2. 네트워크 구조
 - 4.3. Advantages and Disadvantages
5. Conclusion

Diffusion

1. Abstract
2. Introduction
 - 2.1. Diffusion model 정의
 - 2.2. 가우시안 노이즈 설정
 - 2.3. 이전모델과 비교
3. Background
 - 3.1. Reverse process
 - 3.2. Foward(diffusion) process
 - 3.3. Negative log likelihood
 - 3.4. Closed form
 - 3.5. Loss function
 - 3.6. 원본으로 복원
4. Method
 - 4.1. Foward process(L_t)
 - 4.2. Reverse process ($L_1:t-1$)
 - 4.3. Algorithms
 - 4.4. Data scaling
 - 4.5. Simplified training objective
 - 4.6. 결과 비교

Conclusion

GAN

1. Abstract

- Adversarial Process
 - G (Generator) - 실제 데이터의 분포를 학습해 가짜 데이터를 만들어냄
 - D (Discriminator) - 입력받은 샘플이 실제 데이터인지 가짜인지 판별함
- Minmax Game
 - G의 목표 - D가 가짜 데이터라고 잘못 판별할 확률을 최대화
 - D의 목표 - 진짜 데이터와 가짜 데이터를 정확히 구분하는 것

⇒ 서로의 목표가 상충하는 과정에서 성능 발전 유도
- 학습 후
 - G - 실제 데이터와 구분이 불가능한 가짜 데이터 형성
 - D - 판별 확률이 1/2인 상태에 도달
- Backpropagation
 - 기존 생성 모델들은 복잡한 수학적 기법이 필요했으니 GAN은 MLP구조만으로 효율적인 학습 가능

2. Introduction

2.1. 기존 딥러닝 모델

- 복잡한 입력을 레이블 별로 분류하는 discriminative 역할은 뛰어나지만 새로운 데이터를 형성하는 generative 역할은 부족
- 연산이 너무 복잡하고 역전파 적용이 어려움

2.2. Adversarial Nets 제안

- G - 가짜 샘플 생성(MLP)
- D - G가 만든 샘플과 실제 데이터를 판별(MLP)

2.3. Advantages of GAN

- 역전파 + 드롭아웃을 활용한 훈련 가능
- 순전파만으로 샘플링 가능
- 마르코프 체인과 같은 복잡한 연산 필요없음

3. Methods

3.1. Adversarial nets

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

- Pdata(x) - Real Data Distribution
 - 모방하고자 하는 진짜 데이터들의 집합
 - m개의 샘플을 minibatch 학습에 사용
- Pz(z) - Prior on Noise
 - generator가 가짜 데이터를 만들기 위해 사용하는 노이즈 집합
 - m개의 노이즈 샘플을 minibatch 학습에 사용
- Discriminator
 - 입력받은 데이터 x가 진짜 데이터일 확률을 0~1 값으로 출력
 - x에 대해 1에 가까운 값을 G(z)에 대해서는 0에 가까운 값 출력
 - threshold - D(x) = 1/2
- Generator
 - 노이즈 z를 입력받아 x와 비슷한 가짜 데이터를 생성
 - mapping - noise vector를 latent space로 변환
- 역할
 - 입력 : 노이즈 변수 $z \sim P_z(z)$ Generator $G(z; \theta_g) \rightarrow z$ 를 데이터 공간으로 매핑하여 가짜 데이터 샘플을 생성

- Discriminator $D(x; \theta_d) \rightarrow$ 입력 데이터가 실제인지 가짜인지 판별하고 하나의 스칼라 값 출력
- 최종적으로 G는 D를 속이는 데이터를 생성함

$$\mathbb{E}_{x \sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))].$$

- 목적함수

$$\mathbb{E}_{x \sim p_{\text{data}}(x)}[\log D(x)]$$

- 실제 데이터가 들어왔을 때 그 데이터가 실제 데이터일 확률

$$\mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

- 샘플이 실제 데이터인지 아닌지를 판별하는 값(D는 최대화, G는 최소화)

- Problem 1 - **Training imbalance**

- 학습 과정에서 D를 완전히 최적화하려고 하면 계산 비용이 매우 커지고 유한한 데이터에서 overfitting이 발생할 수 있음

→ Generator는 매우 약한 gradient만 받게 되어 학습이 불안정해짐

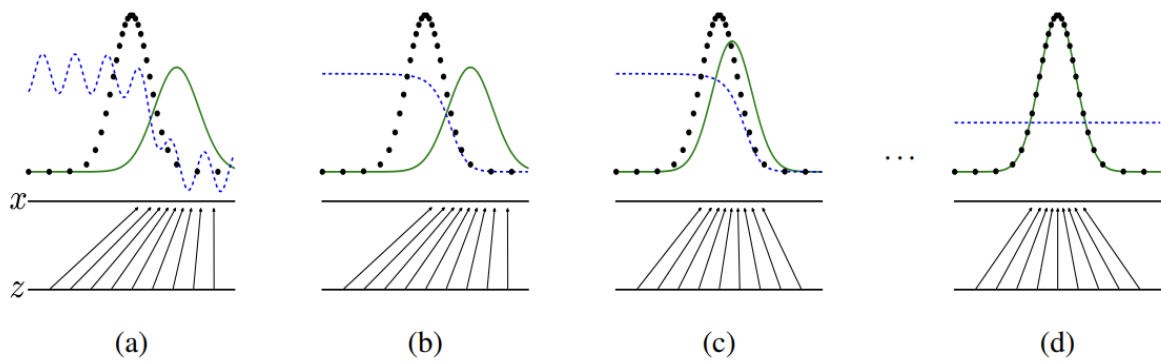
- D를 k번 업데이트, G를 1번 업데이트

→ G가 천천히 학습되는 동안 D를 거의 최적 상태에 가깝게 유지하도록 함

- Problem 2 - **Gradient Saturation**

- $\log(1 - D(G(z)))$ 항이 쉽게 saturation 상태가 되면서 vanishing gradient 문제가 발생

- $\log(1 - D(G(z)))$ 를 최소화하는 대신 $\log D(G(z))$ 를 최대화하도록 학습함



- (a) : 생성 모델의 분포가 실제 데이터 분포와 유사해지고, 판별 모델 D가 부분적으로 정확한 분류기를 형성함
- (b) : 판별 모델 D가 훈련 데이터와 생성된 데이터를 구별하도록 학습됨
- (c) : D의 기울기에 따라 $G(z)$ 가 실제 데이터로 분류될 가능성이 높은 영역으로 이동하도록 학습됨
- (d) : G와 D가 충분한 용량을 가지고 있다면, 학습이 진행될수록 생성 모델의 분포는 실제 데이터 분포에 수렴함

3.2. Theoretical Results

- Problem - 목적함수 불안정
 - 몬테 카를로 기법
 - 2단계로 최적화 나누기
1. Optimize Discriminator first

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D \left(\mathbf{x}^{(i)} \right) + \log \left(1 - D \left(G \left(\mathbf{z}^{(i)} \right) \right) \right) \right].$$

2. Optimize Generator then

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log \left(1 - D \left(G \left(\mathbf{z}^{(i)} \right) \right) \right).$$

- Algorithm 1 - minibatch SGD
 - D 학습 (k번 반복)
 - 실제 데이터 가짜 데이터를 섞어 보여줌
 - Gradient Ascent로 D가 더 잘 판별하도록 파라미터 업데이트
 - G 학습(1번)
 - 가짜 데이터만 생성
 - Gradient Descent로 D의 실수를 유도하는 방향으로 파라미터 업데이트
- Proposition 1

$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$$

- 어떤 데이터 x가 들어왔을때 Pdata에 속할 확률과 Pg에 속할 확률의 비율로 점수 측정
- 1에 가까울수록 진짜 데이터에, 0에 가까울수록 가짜 데이터만 존재
- Theorem 1

$$\begin{aligned} C(G) &= \max_D V(G, D) \\ &= \mathbb{E}_{\mathbf{x} \sim p_{data}} [\log D_G^*(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z} [\log(1 - D_G^*(G(\mathbf{z})))] \\ &= \mathbb{E}_{\mathbf{x} \sim p_{data}} [\log D_G^*(\mathbf{x})] + \mathbb{E}_{\mathbf{x} \sim p_g} [\log(1 - D_G^*(\mathbf{x}))] \\ &= \mathbb{E}_{\mathbf{x} \sim p_{data}} \left[\log \frac{p_{data}(\mathbf{x})}{p_{data}(\mathbf{x}) + p_g(\mathbf{x})} \right] + \mathbb{E}_{\mathbf{x} \sim p_g} \left[\log \frac{p_g(\mathbf{x})}{p_{data}(\mathbf{x}) + p_g(\mathbf{x})} \right] \end{aligned}$$

- Pg = Pdata 일때 D(x) = 1/2
- 목적함수 값 - 전체 비용함수 C(G) = -log (4)

$$C(G) = -\log(4) + KL\left(p_{\text{data}} \left\| \frac{p_{\text{data}} + p_g}{2} \right\| \right) + KL\left(p_g \left\| \frac{p_{\text{data}} + p_g}{2} \right\| \right)$$

- 수학적으로 Jensen-Shannon Divergence(JSD) 최소화
 - Jensen-Shannon Divergence(JSD) $\Rightarrow$ 두 확률 분포 사이 거리 측정하는 지표
 - KL Divergence의 비대칭성 문제 해결

$$D_{JS}(P \parallel Q) = D_{JS}(Q \parallel P)$$

- JSD로 P_g 가 P_{data} 에 안정적으로 도달하도록 함

4. Experiments

4.1. Dataset

- MNIST(손글씨 숫자)
- TFD(얼굴 이미지 데이터)
- CIFAR-10(사물 이미지)

Model	MNIST	TFD
DBN [3]	138 $\pm$ 2	1909 $\pm$ 66
Stacked CAE [3]	121 $\pm$ 1.6	2110 $\pm$ 50
Deep GSN [6]	214 $\pm$ 1.1	1890 $\pm$ 29
Adversarial nets	225 $\pm$ 2	2057 $\pm$ 26

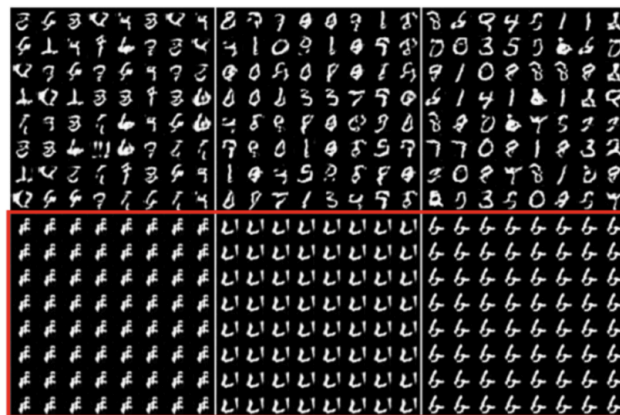
4.2. 네트워크 구조

- G - ReLU와 Sigmoid 활성화 함수를 결합해 활용함
 - 노이즈(입력값)은 네트워크 입력층에만 넣음

- D - Maxout 활성화 함수와 Dropout 적용
- Gaussian Parzen Window(Parzen window 기반 로그 우도 추정치)
 - 수학적으로 likelihood를 직접 계산하기 어려운 GAN 구조
 - G가 만든 샘플에 가우시안 분포를 씌워 전체적인 데이터 밀도 추정
 - 변동성이 크고 고차원 데이터에는 정확도가 떨어짐

4.3. Advantages and Disadvantages

- The challenges
 - Mode collapse - 다양한 이미지를 만드는 법을 학습하는 것이 아닌 가장 자신 있는 하나의 이미지만 계속해서 만들어내는 현상



- GAN은 데이터 생성은 가능하지만 특정 데이터가 나올 확률이 얼마인지는 blackbox 형태로 작동
- The Strengths
 - 마르코프 체인 불필요 - GAN은 한 번의 연산으로 결과 생성 가능
 - 역전파만 사용 - 복잡한 inference 과정 없이 역전파만으로 학습 가능
 - GAN은 매우 선명하고 뚜렷한 이미지 표현 가능

5. Conclusion

- Advanced GANs 제안
 - WGAN - vanishing Gradient, Mode collapse를 해결하기 위해 KL Divergence가 아닌 Wasserstein Distance를 이용함
 - DCGAN - GAN의 단점인 학습이 불안정하다는 것을 극복한 모델, Convolution layer를 이용하여 이미지의 feature map을 추출하고 그것을 GAN모델에 이용함
 - cGAN - 입력값에 조건(c)을 추가하면 원하는 특정 종류의 데이터 형성 가능
-

Diffusion

1. Abstract

- 모델 background
 - 비평형 열역학(nonequilibrium thermodynamics)의 관점에서 비롯된 확산 확률 모델(diffusion probabilistic models)로 고차원 이미지 형성(?)
- Diffusion 모델
 - **training** : 이미지를 노이즈를 섞어 형태를 알아볼 수 없는 상태로 만드는 과정을 학습함
 - **generating** : 노이즈 상태에서 노이즈를 제거해가며 선명한 이미지를 생성함
- Denoising score matching & Langevin 역학
 - 모델이 노이즈를 어떻게 제거할 것인지에 대한 수학적 방법론
 - Denoising score matching(스코어 매칭)
 - 데이터가 어디에 분포해있는지 그 방향을 찾는 것
 - Langevin 역학
 - 노이즈를 조금씩 제거해가며 정답 이미지를 찾아가는 알고리즘

- Weighted Variational Bound(Lsimple)
 - base - ELBO란 (Variational Lower Bound)
 - 완벽하게 특징을 반영하는 데이터 분포를 구하는것은 불가능하므로 하한선을 정해서 근사값을 찾는 것
 - 왜 “weighted”인가 (복잡한 가중치를 생략하고 새롭게 조정했는가)
 - 수식을 그대로 사용하는것보다 특정 단계에 가중치를 다르게 주거나 생략한 수식으로 학습했을 때 이미지 품질이 훨씬 좋아짐
- 자기회귀 디코딩(Autoregressive Decoding)
 - ChatGPT가 문장을 만드는 방식과 비슷함(이전 단계 결과를 보고 다음 단계 결과를 반영하는 순차적 방식)
 - Diffusion 모델에서의 적용
 - 이미지 전체의 노이즈를 단계별로 제거하며 이미지 생성
 - Progressive lossy decompression scheme 방식과 같음
- 결과
 - CIFAR10 데이터셋에서 SOTA FID score 달성

2. Introduction

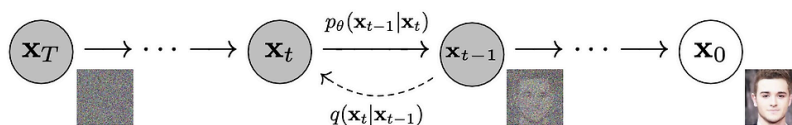


Figure 2: The directed graphical model considered in this work.

2.1. Diffusion model 정의

$$P[s_{t+1}|s_t] = P[s_{t+1}|s_1, \dots, s_t]$$



- 유한한 시간 후에 데이터와 일치하는 샘플을 생성하기 위해 variational inference을 사용해 훈련된 마르코프 체인
 - **마르코프 체인**
 - ⇒ 특정 state의 조건부 확률 분포는 독립적으로 현재 상태에 의존적(다음 상태는 오직 현재 상태에 의해서만 결정)
- **diffusion process**
 - 신호가 파괴될때까지 샘플의 반대방향으로 데이터에 점차적으로 노이즈를 추가하는 마르코프 체인
 - 선명한 사진(X_0)에 노이즈를 섞어 X_1 를 생성 → X_1 에 노이즈를 섞어 X_2 생성
 - ⇒ 다음 단계를 생성하기 위해서는 현재 단계 상태만 필요함(chain)
- **reverse diffusion process**
 - diffusion process를 다시 역행하도록 학습함
 - 노이즈 상태인 X_T 를 보고 직전 단계 X_{T-1} 를 추측
 - ⇒ 다음 단계만 보고 현재 단계를 맞추는 방식(chain)

2.2. 가우시안 노이즈 설정

- 노이즈의 양이 적으면 샘플링 전이를 조건부 가우시안으로 설정해도 충분하다
 - 복잡한 이미지를 한 번에 생성하는 것은 수학적으로 매우 어려우나 적은 양의 노이즈를 제거하는 것은 가우시안 분포 문제로 해결가능함
 - 이미지 생성 모델 ⇒ noise predictor
- 학습 단순화
 - 노이즈를 조금씩만 추가하면 diffusion process와 같이 reverse diffusion process도 가우시안 분포로 모델링 가능
 - 왜 가우시안 분포인가?
 1. 계산적 편의 - 선형 결합이 가능해 평균과 분산만 이용해서 편리하게 계산 가능
 2. 노이즈 제거 = 노이즈 추가 대칭적 구조
 3. 복잡한 이미지 데이터 다루는 물리적 방법
 - 효율적인 신경망 학습이 가능함

2.3. 이전모델과 비교

- GAN, VAE와 비교했을 때 Diffusion은 노이즈를 단계별로 추가, 제거하면서 학습이 매우 안정적이고 고품질의 이미지를 생성함

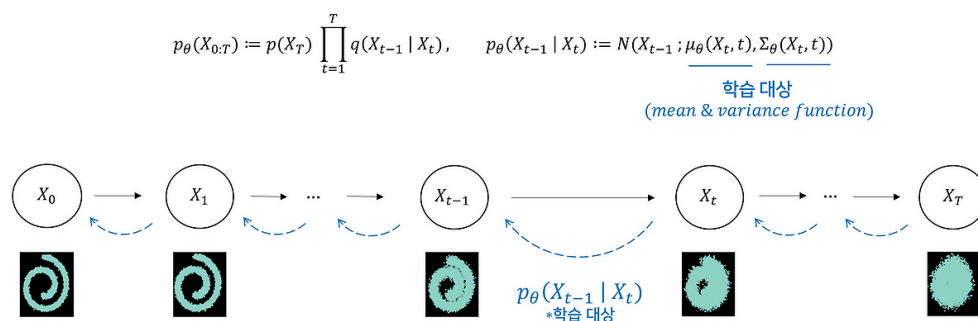
3. Background

3.1. Reverse process

$$p_{\theta}(\mathbf{x}_{0:T}) := p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t),$$

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \Sigma_{\theta}(\mathbf{x}_t, t))$$

- p_{θ} (최종 목표AI) : 완전한 노이즈($p(\mathbf{x}_T)$) $\rightarrow$ $t \sim t-1$ 을 거쳐 가우시안 분포를 따라 이미지를 복원($p(\mathbf{x}_0)$)하는 과정(학습O)
- 노이즈가 섞인 데이터에서 **노이즈를 조금씩 제거하며**, 그 과정을 **p**라 함
 - p는 학습 가능한 파라미터로 이루어짐



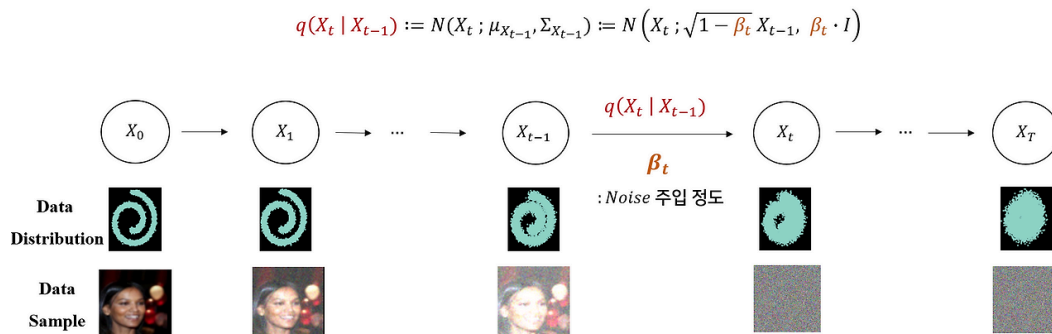
3.2. Forward(diffusion) process

$$q(\mathbf{x}_{1:T} | \mathbf{x}_0) := \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1}),$$

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t I)$$

- q : 원본 사진($\mathbf{x}_0$) $\rightarrow$ variance schedule(β_t)를 따라 가우시안 노이즈를 추가하는 과정(학습X)
- 원본 이미지에 **노이즈를 섞어주며**, 그 과정을 **q**라 표현함

- 논문에서 0.0001~0.02까지 점진적으로 β 를 늘려가며 스케줄링 진행
($\beta t \rightarrow$ 매 스텝 t 에 따라 조금씩 증가하는 상수, 노이즈를 섞는 정도)



3.3. Negative log likelihood

$$\mathbb{E}[-\log p_\theta(\mathbf{x}_0)] \leq \mathbb{E}_q \left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] = \mathbb{E}_q \left[-\log p(\mathbf{x}_T) - \sum_{t \geq 1} \log \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})} \right] =: L$$

- 모델이 이미지를 잘 복원하도록 학습하기 위한 최종 Loss function
- Reverse process가 학습되기 위해 모델이 **training data의 likelihood**를 **최대화하는 방향으로** 학습함
 - **likelihood** $\rightarrow$ AI가 원본이미지(X_0)를 만들어내는 확률
 - 이 likelihood를 직접 계산하기 매우 어려워 위 식을 활용해 이 점수를 잘 받으면 고품질의 이미지를 만들것을 보장함(ELBO)
 - negative log likelihood 대신 **ELBO**를 활용해 그 하한선을 최소화하도록 학습을 진행함
- p와 q연결
 - **p(reverse process)와 q(foward process)**을 하나로 묶어주는 역할
 - p와 q방식이 얼마나 같은지 비교해 그 차이를 환산함

3.4. Closed form

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$$

- 기존 : X100을 만들기 위해서 노이즈를 100번 섞어야함
- Diffusion : 원본(X0)에서 t단계 XT로 한 번에 건너뛰어 학습 속도를 절약

3.5. Loss function

$$\mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \| p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} \underbrace{- \log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0} \right]$$

- 학습을 위해 원본(X0)와 모델 예측 결과를 비교하기 위한 손실함수
- **KL발산(Divergence)** 활용 - q의 가우시안 분포와 p의 가우시안 분포가 얼마나 같은지 측정
- **Lt-1**: q와 p사이의 거리를 계산해 이 거리를 줄이는 방향으로 모델을 학습시킴

3.6. 원본으로 복원

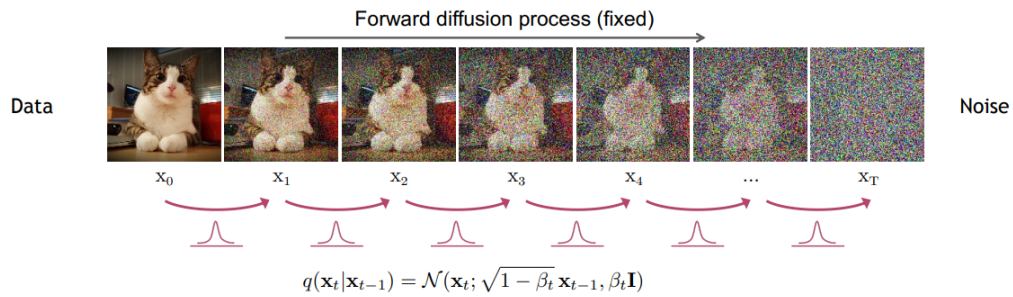
$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t\mathbf{I}),$$

$$\text{where } \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\mathbf{x}_t \quad \text{and} \quad \tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t$$

- 원본(X0)와 현재 이미지(Xt)에 대해 이전 단계(Xt-1)의 평균과 분산을 구하는 식 → 정답
- 이 정답과 같은 평균과 분산을 모델이 생성하도록 학습시킴
- Diffusion 모델의 학습 목적
 - p가 q와 최대한 가까워질 수 있도록 만드는 과정
 - 노이즈가 섞인 데이터에서 원본 데이터를 복구하거나 노이즈만 있는 데이터에서 원하는 이미지를 만들어내는 과정
 - 이때 p와 q의 거리는 KL Divergence 수식을 통해 계산함
 - gradient descent을 활용해 파라미터 업데이트함

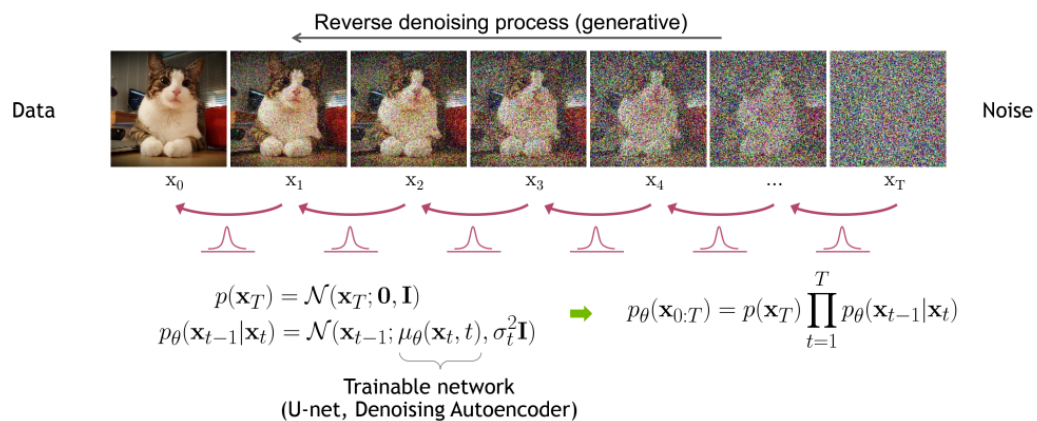
4. Method

4.1. Forward process(Lt)



- 원본 사진에 노이즈를 추가해 의도적으로 망가뜨리는 과정 (q)
- β_t 를 상수로 정해서 학습시키지 않음

4.2. Reverse process (L1:t-1)



- 노이즈에서 사진을 복구하는 과정(p) - ELBO

$$L_{t-1} = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)\|^2 \right] + C$$

- 모델이 예측한 이미지의 평균값(μ_θ)이 수학적으로 계산된 원본의 평균값(μ_t)과 얼마나 가까운지 비교함 - 모델 초기 목표(이미지 복원)

$$L_{t-1} - C = \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{1}{2\sigma_t^2} \left\| \tilde{\mu}_t \left(\mathbf{x}_t(\mathbf{x}_0, \epsilon), \frac{1}{\sqrt{\alpha_t}} (\mathbf{x}_t(\mathbf{x}_0, \epsilon) - \sqrt{1 - \alpha_t} \epsilon) \right) - \mu_\theta(\mathbf{x}_t(\mathbf{x}_0, \epsilon), t) \right\|^2 \right] \quad (9)$$

$$= \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{1}{2\sigma_t^2} \left\| \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t(\mathbf{x}_0, \epsilon) - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon \right) - \mu_\theta(\mathbf{x}_t(\mathbf{x}_0, \epsilon), t) \right\|^2 \right] \quad (10)$$

- 이미지 전체를 한번에 생성하는 것이 아닌 노이즈 값(ϵ)을 찾는 방식으로 모델을 학습하는 것
- $\epsilon_\theta(\mathbf{x}_t, t) \rightarrow$ 특정 시점 t 와 이미지 $\mathbf{x}_t$ 가 주어졌을때 해당 시점의 노이즈(ϵ_t)를 예측하는 네트워크

4.3.Algorithms

Algorithm 1 Training

```

1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
      $\nabla_\theta \|\epsilon - \epsilon_\theta(\sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon, t)\|^2$ 
6: until converged

```

Algorithm 2 Sampling

```

1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 

```

- Algorithm 1 : Training(학습 알고리즘)
 - 모델을 학습시키는 과정
 - 1. 데이터셋에서 아무 원본 사진($\mathbf{x}_0$) 가져오기
 - 2. 여러 단계중 노이즈 섞을 정도를 랜덤하게 선택
(t 가 클수록 이미지에 노이즈가 더 많이 섞임)
 - 3. 이미지에 섞을 정답 노이즈를 생성
 - 4. 모델 학습 후 모델이 예측한 노이즈와 정답 차이를 계산하고 그 차이만큼 모델을 업데이트함

$\Rightarrow$ Closed form으로 원본만 가지고 효율적으로 학습 진행

- Algorithm 2 : Sampling(생성 알고리즘)
 - 학습이 끝난 모델로 이미지를 생성하는 과정
 - 1. 가우시안 노이즈 준비
 - 2. 학습 알고리즘과 반대과정으로 이미지를 복구함

3. 매 단계마다 미세한 랜덤 노이즈(z)를 더해 매번 풍부한 이미지를 생성하도록함 (langevin 역학 실현)
4. 현재 노이즈 - 모델이 예측한 노이즈
5. 최종 결과물 반환

⇒ 노이즈만 가지고 실행해 마르코프 체인 과정으로 진행

4.4. Data scaling

$$p_{\theta}(x_0|x_1) = \prod_{i=1}^D \int_{\delta_{-}(x_0^i)}^{\delta_{+}(x_0^i)} \mathcal{N}(x; \mu_{\theta}^i(x_1, 1), \sigma_1^2) dx$$

- D → 데이터 차원
- i → 이미지 데이터의 픽셀
- 데이터 스케일링
 - {0,1..., 255} 범위의 이미지 데이터를 [-1,1]로 모델이 계산하기 편한 상태로 전처리함
- 모델이 생성한 이미지를 정수 픽셀로 변환해서 저장하는 최종 출력 필터

4.5. Simplified training objective

$$L_{simple}(\theta) := \mathbb{E}_{t,x_0,\epsilon} [\|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2]$$

- 특정 시점의 이미지에 추가된 노이즈를 잘 예측하는 것을 목표로 학습하는 모델
 - 복잡한 가중치를 제거하고 예측 오차만 계산(simple)
 - 기존 ELBO식에서 앞의 복잡한 가중치를 1로 생략
 - t가 작을때(노이즈가 거의 없는 상태) → 학습 강도 낮음
 - t가 클때(노이즈가 많은 상태) → 학습 강도 높음
- ⇒ t가 이미지 형태를 잘 다루게 되어 전체적인 이미지 구도가 더 선명하게 잡힘

4.6. 결과 비교

Table 1: CIFAR10 results. NLL measured in bits/dim.

Model	IS	FID	NLL Test (Train)
Conditional			
EBM [11]	8.30	37.9	
JEM [17]	8.76	38.4	
BigGAN [3]	9.22	14.73	
StyleGAN2 + ADA (v1) [29]	10.06	2.67	
Unconditional			
Diffusion (original) [53]			≤ 5.40
Gated PixelCNN [59]	4.60	65.93	3.03 (2.90)
Sparse Transformer [7]			2.80
PixellQN [43]	5.29	49.46	
EBM [11]	6.78	38.2	
NCSNv2 [56]		31.75	
NCSN [55]	8.87 ± 0.12	25.32	
SNGAN [39]	8.22 ± 0.05	21.7	
SNGAN-DDLS [4]	9.09 ± 0.10	15.42	
StyleGAN2 + ADA (v1) [29]	9.74 ± 0.05	3.26	
Ours (L , fixed isotropic Σ)	7.67 ± 0.13	13.51	≤ 3.70 (3.69)
Ours (L_{simple})	9.46 ± 0.11	3.17	≤ 3.75 (3.72)

Table 2: Unconditional CIFAR10 reverse process parameterization and training objective ablation. Blank entries were unstable to train and generated poor samples with out-of-range scores.

Objective	IS	FID
$\tilde{\mu}$ prediction (baseline)		
L , learned diagonal Σ	7.28 ± 0.10	23.69
L , fixed isotropic Σ	8.06 ± 0.09	13.22
$\ \tilde{\mu} - \tilde{\mu}_\theta\ ^2$	—	—
ϵ prediction (ours)		
L , learned diagonal Σ	—	—
L , fixed isotropic Σ	7.67 ± 0.13	13.51
$\ \epsilon - \epsilon_\theta\ ^2 (L_{\text{simple}})$	9.46 ± 0.11	3.17

- [Table1]
 - IS(Inception Score) - 생성된 이미지가 얼마나 다양하고 뚜렷한가
 - FID - 실제 사진과 생성된 사진의 통계적 거리
 - NLL(Negative Log-Likelihood) - 모델이 데이터의 확률분포를 얼마나 정확하게 맞히는가
 - GAN과 비교했을때 3.17점으로 GAN과 비슷한 수준으로 고품질 이미지 생성 가능을 증명
- [Table2]
 - $u \sim \text{prediction}(\text{baseline})$ - 원본 이미지를 직접 예측하려고 했을때 FID가 13.22~23.69로 별로 좋지 않음
 - ϵ prediction - 이미지 속 노이즈(ϵ)를 예측할때 FID 상승
 - loss function - 수학적으로 정교한 것보다 노이즈의 예측 오차만 계산했을때 (L_{simple}) 가장 좋은 성능을 보임

출처 -

Diffusion-Model-이해하기-with-DDPM

Conclusion

- **Forward Process**는 주어진 데이터에 **노이즈를 조금씩 추가**하여 완전한 노이즈를 만드는 과정
- **Reverse Process**는 완전한 노이즈로부터 거꾸로 **조금씩 노이즈를 걷어내는**, 즉 원래의 데이터를 복원하는 과정이며, 학습을 위한 신경망이 사용